

GOVERNO DO ESTADO DO CEARÁ
SECRETARIA DA EDUCAÇÃO — SEDUC
Educação Profissional • Eixo Informação e Comunicação

LÓGICA DE PROGRAMAÇÃO

Apostila do Estudante

MÓDULO 2 — LINGUAGENS E PYTHON BÁSICO

Aulas 11 a 20

Tipos de Linguagem • Python • Fundamentos do Algoritmo • Compilação e Interpretação

E.E.E.P. Lúcia Helena Viana Ribeiro — Horizonte/CE
Professor Francisco Ericson Cornélio da Costa

Sumário

Sumário.....	2
Apresentação.....	3
Unidade 1 — Tipos de Linguagens de Programação.....	4
1.1 Da linguagem de máquina às linguagens de alto nível	4
1.2 Linguagem estruturada e linguagem orientada a objetos.....	4
1.3 Por que aprender Python	4
Unidade 2 — Python: Conceitos Básicos	6
2.1 O que é o Python e como executamos	6
2.2 O primeiro programa: o comando print.....	6
2.3 Comentários	6
2.4 Regras de sintaxe que você não pode esquecer	6
2.5 Erros comuns de quem está começando	6
Unidade 3 — Fundamentos da Construção do Algoritmo	9
3.1 A estrutura geral de um algoritmo	9
3.2 Blocos e indentação.....	9
3.3 As três estruturas básicas	9
3.4 Boas práticas na construção	9
Unidade 4 — Programa e Execução: Compilação e Interpretação.....	11
4.1 Código-fonte e programa	11
4.2 Compilação.....	11
4.3 Interpretação.....	11
4.4 Vantagens e desvantagens	11
Unidade 5 — Aplicação dos Métodos em Portugol e Python	13
5.1 O mesmo algoritmo em duas linguagens.....	13
5.2 Um exemplo com três linhas	13
Gabarito Comentado.....	16
Unidade 1.....	16
Unidade 2.....	16
Unidade 3.....	16
Unidade 4.....	17
Unidade 5.....	17

Apresentação

No Módulo 1 você aprendeu a pensar de forma lógica: algoritmos, raciocínio, descrição narrativa e fluxograma. Agora damos o próximo passo: entender como esses algoritmos viram programas de verdade dentro do computador.

Neste módulo você vai conhecer os **tipos de linguagem de programação**, ter o primeiro contato com o **Python**, entender como um programa é executado (a diferença entre **compilar** e **interpretar**) e praticar a escrita do mesmo algoritmo em Portugol e em Python.

✓Como usar esta apostila

Leia a teoria, refaça os exemplos de código no computador e resolva todos os exercícios. Programar se aprende digitando: sempre que houver um trecho de código, teste-o você mesmo. Confira o Gabarito Comentado ao final só depois de tentar.

Bons estudos!

Unidade 1 — Tipos de Linguagens de Programação

Aula correspondente: Aula 11 — Tipos de Linguagens (Estruturada e Orientada a Objetos).

1.1 Da linguagem de máquina às linguagens de alto nível

O computador, no fundo, só entende dois valores: 0 e 1. Essa é a **linguagem de máquina** (código binário). Escrever programas diretamente em 0 e 1 seria quase impossível para um humano. Por isso foram criadas linguagens cada vez mais próximas da nossa forma de escrever.

Chamamos de **baixo nível** as linguagens próximas da máquina (como o Assembly) e de **alto nível** as linguagens próximas da linguagem humana (como Python, Java e C). Quanto mais alto o nível, mais fácil para a pessoa e mais distante dos 0 e 1.

Ideia central

Escrevemos em uma linguagem de alto nível (que entendemos) e um programa tradutor converte isso para a linguagem de máquina (que o computador entende).

1.2 Linguagem estruturada e linguagem orientada a objetos

Entre as linguagens de alto nível, dois paradigmas (formas de organizar o programa) se destacam:

- **Programação estruturada:** o programa é uma sequência de instruções e sub-rotinas, organizada por três estruturas básicas — sequência, decisão e repetição. É o estilo do Portugol e de linguagens como C.
- **Programação orientada a objetos (POO):** o programa é organizado em objetos, que reúnem dados (atributos) e ações (métodos). Um objeto "Aluno", por exemplo, teria atributos como nome e nota, e métodos como calcular média. É o estilo de linguagens como Java.

O **Python** é uma linguagem multiparadigma: permite programar de forma estruturada (que é como começaremos) e também orientada a objetos. Neste curso partimos do estilo estruturado, que dialoga diretamente com a lógica que você já aprendeu.

1.3 Por que aprender Python

Python tem sintaxe simples e legível, é uma das linguagens mais usadas do mundo (em web, análise de dados e inteligência artificial) e é excelente para quem está começando, porque exige menos código para fazer o mesmo que outras linguagens.

Exercícios — Unidade 1

1. Marque V (verdadeiro) ou F (falso):

- () O computador entende diretamente a linguagem Python, sem tradução.
- () Linguagem de máquina é formada por 0 e 1.
- () Assembly é uma linguagem de alto nível.
- () Quanto mais alto o nível da linguagem, mais próxima ela está do ser humano.

2. Explique, com suas palavras, a diferença entre linguagem de baixo nível e de alto nível.

3. Qual a diferença entre programação estruturada e orientada a objetos? Dê a ideia central de cada uma.

4. Em orientação a objetos, classifique cada item como ATRIBUTO ou MÉTODO de um objeto "Carro":

- a) cor ()
- b) acelerar ()
- c) velocidade ()
- d) frear ()

5. Por que é mais fácil para uma pessoa escrever um programa em Python do que em linguagem de máquina?

6. Cite duas áreas em que Python é bastante utilizado atualmente.

7. O Python permite programar apenas de forma orientada a objetos? Justifique.

Unidade 2 — Python: Conceitos Básicos

Aulas correspondentes: Aulas 12 e 13 — Python: conceitos básicos da linguagem.

2.1 O que é o Python e como executamos

Python é a linguagem que usaremos no curso. Escrevemos o código em um arquivo de texto com extensão **.py** e um programa chamado **interpretador** lê esse arquivo e executa as instruções. Você pode escrever e rodar Python em editores como o VS Code, no IDLE (que vem com o Python) ou no modo interativo do terminal.

2.2 O primeiro programa: o comando print

O comando **print** serve para mostrar algo na tela (a saída do programa). O texto a ser mostrado vai entre aspas, dentro dos parênteses:

Código Python:

```
print("Olá, mundo!")
```

Saída na tela:

Olá, mundo!

Podemos mostrar números (sem aspas) e o Python até calcula expressões numéricas antes de mostrar:

Código Python:

```
print(10 + 5)
```

Saída na tela:

15

⚠️ Texto entre aspas não é calculado

`print("10 + 5")` mostra **10 + 5** (o texto, como está), enquanto `print(10 + 5)` mostra **15** (o resultado da conta). As aspas mudam tudo.

2.3 Comentários

Comentários são anotações no código que o Python ignora ao executar. Servem para explicar o programa a quem o lê. Em Python, um comentário começa com o símbolo **#**:

Código Python:

```
# Este programa cumprimenta o usuário  
print("Bom dia!")
```

2.4 Regras de sintaxe que você não pode esquecer

- **Aspas:** todo texto vai entre aspas. Abriu, tem que fechar.
- **Parênteses:** o print usa parênteses. Abriu, tem que fechar.
- **Maiúsculas e minúsculas importam:** Python diferencia. `print` funciona; `Print` ou `PRINT` dão erro.
- **Uma instrução por linha:** o comum é escrever um comando por linha.

2.5 Erros comuns de quem está começando

Errar faz parte. Veja os erros mais frequentes e como corrigi-los:

Cuidado com estes erros:

```
Print("Oi")      # ERRO: P maiúsculo. Correto: print("Oi")
print("Oi")      # ERRO: faltou fechar as aspas
print("Oi"      # ERRO: faltou fechar o parêntese
print(Oi)        # ERRO: texto sem aspas
```

Exercícios 2A — Ler código e prever a saída

Diga exatamente o que cada programa mostra na tela.

8.

```
print("Estou aprendendo Python")
```

9.

```
print(7 + 3)
```

10.

```
print("7 + 3")
```

11.

```
print("Linha 1")
print("Linha 2")
```

12.

```
# print("Oculto")
print("Visível")
```

Exercícios 2B — Encontrar e corrigir o erro

Cada linha tem um erro. Reescreva-a corretamente.

13.

```
Print("Bom dia")
```

14.

```
print("Bom dia)
```

15.

```
print("Bom dia")
```

16.

```
print(Bom dia)
```

Exercícios 2C — Escrever código

17. Escreva um programa em Python que mostre na tela a frase: Eu gosto de programar.

18. Escreva um programa que mostre seu nome em uma linha e sua cidade na linha seguinte (use dois print).

19. Escreva um programa que mostre o resultado da conta 25×4 (faça o Python calcular, não escreva 100 direto).

20. Escreva um programa com um comentário explicando o que ele faz e um print que mostre a frase: Meu primeiro programa.

Unidade 3 — Fundamentos da Construção do Algoritmo

Aulas correspondentes: Aulas 14 e 15 — Fundamentos da Construção do Algoritmo.

3.1 A estrutura geral de um algoritmo

Todo algoritmo bem construído tem um começo e um fim claros, e um corpo com as instruções no meio. Em pseudocódigo (Portugol), essa estrutura costuma aparecer assim:

Estrutura em Portugol:

```
algoritmo "MeuAlgoritmo"
inicio
    // instruções vêm aqui, uma por linha
finalgoritmo
```

A palavra depois de **algoritmo** é apenas um nome. Tudo o que o algoritmo faz fica entre **inicio** e **finalgoritmo**.

✓A sintaxe exata pode variar

Dependendo da ferramenta usada em aula (VisualG, Portugol Studio e outras), as palavras podem mudar um pouco. O importante é a ideia: nome, início, corpo, fim. Siga o padrão que seu professor adotar no laboratório.

3.2 Blocos e indentação

Um **bloco** é um conjunto de instruções que andam juntas. Para deixar claro quais instruções pertencem a um bloco, usamos a **indentação** (o recuo, aquele espaço no início da linha). No Python, a indentação não é só organização: ela é obrigatória e define os blocos. Já no Módulo 1, ao recuar os passos de um SE/SENÃO, você usava essa ideia.

3.3 As três estruturas básicas

Qualquer algoritmo, por mais complexo, é construído com apenas três tipos de estrutura. Você as estudará em detalhe nos próximos módulos; aqui basta reconhecê-las:

- **Sequência:** as instruções são executadas uma após a outra, de cima para baixo.
- **Decisão (seleção):** o algoritmo escolhe um caminho conforme uma condição (o SE/SENÃO).
- **Repetição:** um trecho é repetido várias vezes enquanto uma condição for válida.

3.4 Boas práticas na construção

Um algoritmo bem escrito é fácil de ler e de corrigir. Use nomes claros, escreva um passo por linha, indente os blocos e comente as partes importantes. Antes de programar, lembre-se sempre do modelo Entrada → Processamento → Saída, do Módulo 1.

Exercícios — Unidade 3

21. No algoritmo em Portugol abaixo, indique qual é o NOME, onde começa e onde termina o corpo:

```
algoritmo "Saudacao"
inicio
    escreval("Olá")
```

```
fimalgoritmo
```

22. Classifique cada situação como SEQUÊNCIA, DECISÃO ou REPETIÇÃO:

- a) Somar dois números e mostrar o resultado. ()
- b) Mostrar "aprovado" se a nota for maior que 6. ()
- c) Escrever os números de 1 a 100. ()
- d) Ler o nome e depois a idade de uma pessoa. ()

23. O que é indentação e por que ela é especialmente importante no Python?

24. Cite três boas práticas na construção de um algoritmo.

25. Reescreva a sequência de passos abaixo de forma organizada, um passo por linha e com nomes claros: "lernumero calcular dobro mostrar".

Unidade 4 — Programa e Execução: Compilação e Interpretação

Aulas correspondentes: Aulas 16 e 17 — Programa e Execução; Métodos de Tradução: compilação e interpretação.

4.1 Código-fonte e programa

O texto que escrevemos em uma linguagem de programação é o **código-fonte**. Mas o computador não executa o código-fonte diretamente: ele precisa que esse código seja traduzido para a linguagem de máquina. Existem dois métodos principais de tradução: a compilação e a interpretação.

4.2 Compilação

Na **compilação**, um programa tradutor chamado **compilador** lê todo o código-fonte de uma só vez e gera um novo arquivo: o programa executável, já em linguagem de máquina. Depois disso, o executável roda sozinho, sem precisar do código-fonte nem do compilador. A linguagem C é um exemplo clássico de linguagem compilada.

4.3 Interpretação

Na **interpretação**, um programa tradutor chamado **interpretador** lê e executa o código-fonte instrução por instrução, na hora, sem gerar um executável separado. Toda vez que o programa roda, o interpretador precisa estar presente. O **Python** é uma linguagem interpretada.

A diferença em uma frase

Compilar é traduzir tudo primeiro e rodar depois; interpretar é traduzir e rodar ao mesmo tempo, linha por linha.

4.4 Vantagens e desvantagens

Compilada (ex.: C)	Interpretada (ex.: Python)
Executa mais rápido, pois já está traduzida.	Mais fácil de testar: roda o código na hora.
Gera um executável que roda sem o tradutor.	Mais portátil entre sistemas diferentes.
Precisa recompilar a cada alteração.	Costuma executar mais devagar.
O executável é específico do sistema.	Precisa do interpretador instalado para rodar.

✓Curiosidade honesta sobre o Python

Na prática, o Python primeiro converte seu código em um formato intermediário (chamado bytecode) e só então o executa. Para o nosso nível de estudo, tratamos o Python como uma linguagem interpretada — o que é a forma como ele se comporta para você.

Exercícios — Unidade 4

26. Marque V (verdadeiro) ou F (falso):

- () O computador executa o código-fonte diretamente, sem tradução.
- () O compilador traduz todo o código de uma vez e gera um executável.
- () O interpretador executa o código instrução por instrução.
- () Python é uma linguagem compilada que gera um executável independente.

Unidade 5 — Aplicação dos Métodos em Portugol e Python

Aulas correspondentes: Aulas 18, 19 e 20 — Aplicação dos Métodos de Tradução em Portugol e Python.

5.1 O mesmo algoritmo em duas linguagens

Uma mesma ideia pode ser escrita em Portugol (para pensar) e em Python (para executar de verdade). Compare o programa que mostra uma saudação nas duas linguagens:

Portugol:

```
algoritmo "Saudacao"
inicio
    escreval("Olá, mundo!")
fimalgoritmo
```

Python:

```
print("Olá, mundo!")
```

Repare: em Portugol usamos **escreval** para mostrar na tela; em Python usamos **print**. A ideia ("mostrar um texto") é a mesma; muda a forma de escrever.

5.2 Um exemplo com três linhas

Portugol:

```
algoritmo "Menu"
inicio
    escreval("1 - Iniciar")
    escreval("2 - Opções")
    escreval("3 - Sair")
fimalgoritmo
```

Python:

```
print("1 - Iniciar")
print("2 - Opções")
print("3 - Sair")
```

Saída (igual nas duas):

```
1 - Iniciar
2 - Opções
3 - Sair
```

✓ Como estudar esta unidade

Digite os dois programas: o Portugol na ferramenta do laboratório e o Python no editor. Rodar os dois e ver a mesma saída deixa claro que a lógica é a mesma — só muda a tradução.

Exercícios 5A — Traduzir de Portugol para Python

32. Traduza para Python:

```
algoritmo "Ola"
inicio
    escreval("Boa tarde!")
fimalgoritmo
```

33. Traduza para Python:

```
algoritmo "Contas"
inicio
    escreval("Resultado:")
    escreval(8 + 2)
fimalgoritmo
```

34. Traduza para Python (três mensagens):

```
algoritmo "Aviso"
inicio
    escreval("Atenção")
    escreval("Prova amanhã")
    escreval("Estudem!")
fimalgoritmo
```

Exercícios 5B — Traduzir de Python para Portugol**35. Traduza para Portugol:**

```
print("Bem-vindo!")
```

36. Traduza para Portugol:

```
print("Total:")
print(15 * 2)
```

Exercícios 5C — Prever a saída e desafios**37. Qual a saída do programa Python abaixo?**

```
print("Nome:")
print("Maria")
print("Idade:")
```

```
print(15)
```

38. O programa Portugol abaixo e um programa Python produzem a mesma saída? Escreva a saída dos dois.

```
algoritmo "Soma"  
inicio  
    escreval(10 + 20)  
fimalgoritmo  
  
# Python  
print(10 + 20)
```

39. Escreva, em Python E em Portugol, um programa que mostre as três frases: "Curso de Lógica", "Modulo 2" e "Python".

40. Desafio: escreva um programa em Python que mostre a seguinte saída exata (duas linhas):

```
Produto: Caderno  
Preço: 12
```

Gabarito Comentado

Tente resolver antes de consultar. O comentário explica o raciocínio, não apenas a resposta.

Unidade 1

1. **F, V, F, V.** O computador não entende Python diretamente, precisa de tradução (F); linguagem de máquina é 0 e 1 (V); Assembly é de baixo nível (F); quanto mais alto o nível, mais próximo do humano (V).
2. Baixo nível é a linguagem próxima da máquina (0 e 1, ou Assembly), difícil para a pessoa. Alto nível é próxima da linguagem humana (Python, Java, C), mais fácil de escrever e entender.
3. Estruturada organiza o programa em sequência de instruções e sub-rotinas, com sequência, decisão e repetição. Orientada a objetos organiza o programa em objetos que reúnem atributos (dados) e métodos (ações).
4. a) cor = ATRIBUTO; b) acelerar = MÉTODO; c) velocidade = ATRIBUTO; d) frear = MÉTODO. Atributos são características (dados); métodos são ações.
5. Porque Python é de alto nível, com comandos próximos da linguagem humana (como print), enquanto a linguagem de máquina usa apenas 0 e 1, praticamente impossível de escrever à mão.
6. Exemplos válidos: desenvolvimento web, análise de dados, inteligência artificial, automação. (Duas quaisquer.)
7. Não. Python é multiparadigma: permite tanto o estilo estruturado quanto o orientado a objetos. Começaremos pelo estruturado.

Unidade 2

8. Estou aprendendo Python
9. 10. O Python calcula $7 + 3$ porque são números sem aspas.
10. $7 + 3$. Como está entre aspas, é texto e aparece exatamente assim, sem ser calculado.
11. Duas linhas: Linha 1 e, abaixo, Linha 2. Cada print mostra em uma linha.
12. Visível. A primeira linha é um comentário (começa com #), então é ignorada; só o segundo print aparece.
13. `print("Bom dia")` — o comando deve ser escrito todo em minúsculas.
14. `print("Bom dia")` — faltava fechar as aspas do texto.
15. `print("Bom dia")` — faltava fechar o parêntese.
16. `print("Bom dia")` — o texto precisa estar entre aspas.
17. `print("Eu gosto de programar.")`
18. Exemplo: `print("Seu Nome")` e, na linha seguinte, `print("Sua Cidade")`. São dois comandos print, um por linha.
19. `print(25 * 4)` — deve mostrar 100, calculado pelo Python.
20. `# mostra uma mensagem\nprint("Meu primeiro programa.")` — a primeira linha é o comentário; a segunda mostra a frase.

Unidade 3

21. NOME: Saudacao. O corpo começa na linha "inicio" e termina na linha "fimalgoritmo"; a instrução escreval fica entre eles.
22. a) SEQUÊNCIA; b) DECISÃO; c) REPETIÇÃO; d) SEQUÊNCIA. Escrever de 1 a 100 repete uma ação; mostrar algo conforme a nota é uma decisão.
23. Indentação é o recuo (espaço) no início da linha, usado para mostrar quais instruções formam um bloco. No Python ela é obrigatória: é a indentação que define os blocos; errá-la causa erro.
24. Exemplos: usar nomes claros; escrever um passo por linha; indentar os blocos; comentar as partes importantes; pensar em Entrada→Processamento→Saída antes de escrever. (Três quaisquer.)

25. 1) ler número; 2) calcular o dobro; 3) mostrar o resultado. Cada ação em uma linha, com nomes claros.

Unidade 4

26. **F, V, V, F.** O código-fonte precisa ser traduzido (F); o compilador traduz tudo e gera executável (V); o interpretador executa instrução por instrução (V); Python é interpretada, não compilada em executável independente (F).

27. Compilar é traduzir todo o código de uma vez, gerando um executável que roda depois. Interpretar é traduzir e executar o código ao mesmo tempo, instrução por instrução, sem gerar executável separado.

28. Compilada. Gerar um executável que roda sozinho, sem o tradutor, é característica da compilação.

29. Vantagem: é mais fácil de testar e mais portátil. Desvantagem: costuma executar mais devagar e precisa do interpretador instalado.

30. Porque Python é interpretado: não gera um executável independente. Para rodar o código, o interpretador Python precisa estar presente naquele computador.

31. a) CÓDIGO-FONTE (o arquivo .py que você escreveu); b) PROGRAMA EXECUTÁVEL (arquivo em linguagem de máquina gerado pelo compilador).

Unidade 5

32. `print("Boa tarde!")`

33. `print("Resultado:")` e, na linha seguinte, `print(8 + 2)`. A saída será: Resultado: e depois 10.

34. `print("Atenção")`, `print("Prova amanhã")`, `print("Estudem!")` — cada frase em um `print`.

35. algoritmo "BemVindo" / inicio / escreval("Bem-vindo!") / fimalgoritmo. (O nome do algoritmo pode ser qualquer um.)

36. algoritmo "Total" / inicio / escreval("Total:") / escreval(15 * 2) / fimalgoritmo. A saída mostra Total: e depois 30.

37. Quatro linhas: Nome: / Maria / Idade: / 15.

38. Sim, produzem a mesma saída: 30. Tanto `escreval(10 + 20)` quanto `print(10 + 20)` calculam a soma e mostram 30.

39. Python: `print("Curso de Lógica")`, `print("Modulo 2")`, `print("Python")`. Portugol: algoritmo "Frases" / inicio / escreval("Curso de Lógica") / escreval("Modulo 2") / escreval("Python") / fimalgoritmo.

40. `print("Produto: Caderno")` e, na linha seguinte, `print("Preço: 12")`. Como tudo está entre aspas, aparece exatamente como escrito.

— Fim do Módulo 2 —

Próximo: Módulo 3 — Variáveis, Tipos de Dados e Operadores (Aulas 21–40)